Black Cat Nails Móvil: Integración de Firebase Authentication y Cloud Firestore en una Aplicación React Native para la Gestión de Citas de un Emprendimiento de Manicura

Enrique Alejandro García Villeda (GV200136)

Facultad de Ingeniería, Universidad Don Bosco

DPS941: Diseño y Programación de Software Multiplataforma, G01T

Alexander Alberto Siguenza Campos

8 de noviembre de 2026

Introducción

Este documento acompaña el Avance 2 de la Etapa 3 del proyecto de cátedra, entregado como Foro de Discusión 2. En el Avance 1 dejé funcionando la aplicación móvil de Black Cat Nails en React Native: inicio de sesión, galería, agendamiento en cuatro pasos y la agenda de la administradora (García Villeda, 2026b). Todo eso corría sobre una base de datos simulada que vivía en el teléfono. Si la clienta borraba la app, perdía sus citas. Y la propietaria no veía nada de lo que pasaba en el teléfono de cada clienta, que era justamente lo que tenía que resolver.

En este avance reemplacé esa capa simulada por Firebase. Las cuentas ahora viven en Firebase Authentication, con correo y contraseña o con Google, y los datos del negocio viven en Cloud Firestore, asociados al usuario autenticado. Aproveché el cambio para agregar los módulos que el proyecto necesitaba y que no tenían sentido sin una base de datos remota: el historial de actividad de cada clienta, los favoritos sincronizados, el inventario de la administradora y las reglas de seguridad que impiden que una clienta lea o modifique lo que no le corresponde.

El documento sigue el orden en que tomé las decisiones. Primero explico qué es Firebase y por qué lo usé de la forma en que lo usé. Después describo la configuración en React Native, el flujo de autenticación, el modelo de datos, las reglas de seguridad y los módulos nuevos. Cierro con las pruebas, un análisis de lo que salió bien y de lo que no me convence del todo, y las recomendaciones para la entrega final.

Firebase en el Proyecto

Qué es Firebase

Firebase es una plataforma de Google que ofrece servicios de backend listos para usar: autenticación, bases de datos, almacenamiento de archivos, funciones en la nube, analítica y mensajería, entre otros. En lugar de programar y mantener un servidor propio, la aplicación habla directamente con esos servicios a través de un SDK, y la seguridad se define con reglas declarativas que se evalúan en los servidores de Google (Firebase, s.f.-a). Moroney (2017) lo describe como un backend como servicio pensado para que un equipo pequeño pueda publicar aplicaciones móviles sin administrar infraestructura.

Para este avance usé dos servicios. Firebase Authentication maneja la identidad: registra cuentas, verifica contraseñas, emite tokens firmados y conecta proveedores externos como Google (Firebase, s.f.-b). Cloud Firestore es una base de datos NoSQL orientada a documentos. Guarda la información en documentos con campos, agrupados en colecciones, y permite escuchar cambios en tiempo real (Firebase, s.f.-c).

Por Qué Firebase y No el Backend de la Etapa 2

La versión web de la Etapa 2 tiene sus propios Route Handlers en Next.js. Pude haber hecho que la app móvil consumiera esa API, pero me pareció peor idea por tres motivos concretos. El primero es la sesión: la web firmaba un JWT de ocho horas en una cookie, y en móvil habría tenido que reimplementar el almacenamiento seguro del token y la renovación. Firebase Auth ya resuelve eso y además trae el inicio con Google, que el Foro 2 pide. El segundo es el tiempo real. La propietaria confirma citas desde su teléfono y la clienta quiere enterarse en ese momento, no cuando recargue. Con Firestore eso es un `onSnapshot`; con una API REST habría necesitado sondeo o WebSockets. El tercero es práctico: soy el único desarrollador y el plan gratuito Spark cubre de sobra el volumen de un salón con un par de cientos de clientas (Firebase, s.f.-h).

Lo que sí conservé de la Etapa 2 es la lógica de negocio. Los módulos de `domain/` (disponibilidad, máquina de estados de la cita, consumo de insumos y retoque) son funciones puras

y siguen corriendo igual. Firebase solo reemplazó la capa de datos, que era lo que había planeado desde el Avance 1.

SDK Web Modular frente a React Native Firebase

Hay dos formas de usar Firebase en React Native. La primera es el SDK de JavaScript oficial, el mismo de la web, en su versión modular. La segunda es la biblioteca React Native Firebase, que envuelve los SDK nativos de Android e iOS. Expo documenta las dos y aclara que la segunda obliga a compilar la aplicación, porque no funciona dentro de Expo Go (Expo, s.f.-a).

Elegí el SDK de JavaScript, versión 13. Tenía una condición desde el Avance 1: que la app se pudiera probar con Expo Go escaneando un código QR. Además, ese mismo SDK es el que usará la versión web en la entrega final, así que los servicios que escribí para el móvil se pueden reutilizar casi sin cambios. El costo es que la caché sin conexión en React Native es solo en memoria, porque el SDK web necesita IndexedDB para persistirla, y eso lo retomo en el análisis.

Configuración de Firebase en React Native

Proyecto, Aplicaciones Registradas y Variables de Entorno

En la consola de Firebase creé el proyecto, activé los proveedores de correo y de Google en Authentication y creé la base de Firestore en modo producción, que arranca con todo bloqueado. Registré una aplicación web, de donde salen los valores de `firebaseConfig`, y una aplicación iOS con el bundle ID `com.enriqueagv.blackcatnails`, que genera el cliente OAuth que necesita Google en el teléfono.

Esos valores no van escritos en el código. Los leo de un archivo `.env` que está en `.gitignore`, y el repositorio solo trae `.env.example` con los nombres. Expo inyecta en el bundle únicamente las variables que empiezan con `EXPO_PUBLIC_` (Expo, s.f.-c). Vale aclarar algo que al principio no tenía claro: la `apiKey` de Firebase no es un secreto. Identifica al proyecto, y cualquiera que descargue la app la puede extraer. Lo que protege los datos son las reglas de seguridad, no esconder esa clave. La única credencial secreta del proyecto es la cuenta de servicio que usa el script de carga de datos, y esa nunca entra a la aplicación.

Inicialización por Plataforma

La inicialización está en `src/services/firebase/`. Firestore se crea con `initializeFirestore`, con la detección automática de long polling activada porque algunas redes móviles cortan los canales de streaming. Authentication es distinto en cada plataforma. El SDK web no sabe dónde guardar la sesión en un teléfono, así que en iOS y Android le paso AsyncStorage con `getReactNativePersistence`. Para no llenar el código de condiciones usé las extensiones de plataforma de Metro: `auth.native.ts` se carga en el teléfono y `auth.ts` en la web. Hice lo mismo con Google Sign-In.

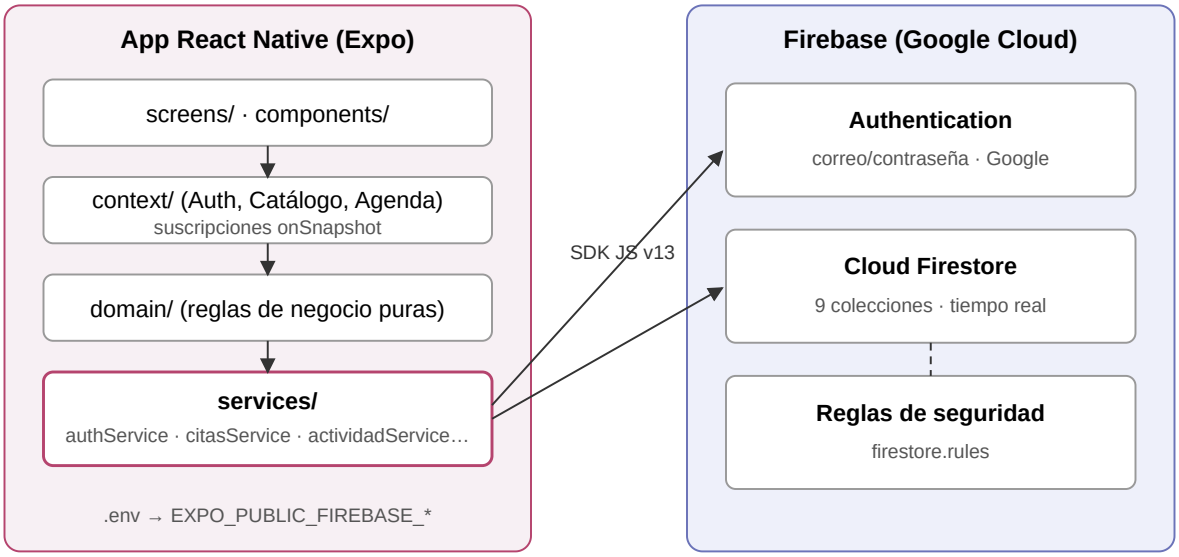
```
// auth.native.ts
auth = initializeAuth(obtenerApp(), {
  persistence: getReactNativePersistence(AsyncStorage),
});
if (usarEmuladores) connectAuthEmulator(auth, `http://${host}:9099`);
```

La última línea conecta la app al Firebase Local Emulator Suite cuando así lo indica el `.env`. Los emuladores levantan Authentication y Firestore en la computadora, con las mismas

reglas de seguridad (Firebase, s.f.-f). Hice todo el desarrollo y las pruebas contra ellos antes de tocar el proyecto real, así que nunca tuve que borrar datos de prueba de la nube.

Figura 1

Arquitectura de la aplicación móvil con Firebase



Nota. Elaboración propia. Las pantallas nunca llaman a Firebase directamente: pasan por los contextos y estos por `services/`. Las reglas se evalúan en el servidor en cada lectura y escritura.

Autenticación con Firebase

Correo y Contraseña

El registro conserva las validaciones Zod del Avance 1, que son las mismas de la web: nombre de tres a sesenta letras, teléfono de ocho dígitos y contraseña de al menos ocho caracteres con letras y números. Después de validar, llamo a `createUserWithEmailAndPassword`. Pero la cuenta de Firebase no alcanza: el negocio necesita teléfono, rol, contador de inasistencias y código de referido. Eso va en el documento `usuarias/{uid}`, que creo en una escritura atómica (batch) junto con la reserva del código de referido. Si esa escritura falla, borro la cuenta recién creada en Authentication para que no quede una usuaria sin perfil.

El inicio de sesión usa `signInWithEmailAndPassword`. Firebase responde con códigos como `auth/invalid-credential` o `auth/too-many-requests`, que están en inglés y pensados para programadores. Escribí una función que los traduce a mensajes para la clienta (Tabla 1). Para credenciales incorrectas uso el mismo mensaje sin importar si el correo existe, y lo mismo hago al pedir la recuperación de contraseña con `sendPasswordResetEmail`: así nadie puede averiguar qué correos están registrados, que es uno de los fallos de autenticación que OWASP señala para apps móviles (OWASP Foundation, 2024).

Tabla 1

Traducción de errores de Firebase Authentication

Código de Firebase	Mensaje que ve la clienta
`auth/invalid-credential`	Correo o contraseña incorrectos
`auth/email-already-in-use`	Ya existe una cuenta con ese correo
`auth/too-many-requests`	Demasiados intentos fallidos. Esperá unos minutos o restablecé tu contraseña.
`auth/network-request-failed`	Sin conexión. Revisá tu internet e intentá de nuevo.
`auth/popup-closed-by-user`	(no se muestra; la clienta cerró la ventana a propósito)
`auth/requires-recent-login`	Por seguridad, cerrá sesión y volvé a entrar antes de cambiar tu correo.
`permission-denied` (Firestore)	No tenés permiso para realizar esta acción

Nota. Elaboración propia con base en `src/services/errores.ts`.

Inicio de Sesión con Google

Google fue la parte que más me costó, porque funciona distinto en cada plataforma. En la web uso `signInWithPopup`, que abre la ventana de selección de cuenta de Google (Firebase, s.f.-g). En el teléfono eso no existe. Ahí uso la biblioteca `@react-native-google-signin/google-signin`, que abre el selector de cuentas del sistema y devuelve un `idToken` firmado por Google. Ese token lo convierto en una credencial de Firebase con `GoogleAuthProvider.credential` y entro con `signInWithCredential` (React Native Google Sign In, s.f.). A partir de ahí la sesión la maneja Firebase igual que si hubiera entrado con correo.

El problema es que esa biblioteca tiene código nativo y no está dentro de Expo Go. Expo lo dice sin rodeos: para Google en el teléfono hace falta una development build (Expo, s.f.-b). No quise que eso rompiera la app en Expo Go, así que cargo el módulo con `require()` recién cuando la clienta toca el botón, y si detecto que corre en Expo Go muestro un mensaje que le pide usar su correo. En iOS la app se compila con `npx expo run:ios`, y el esquema de URL que necesita Google se agrega desde `app.config.ts`, que lo lee del `.env`.

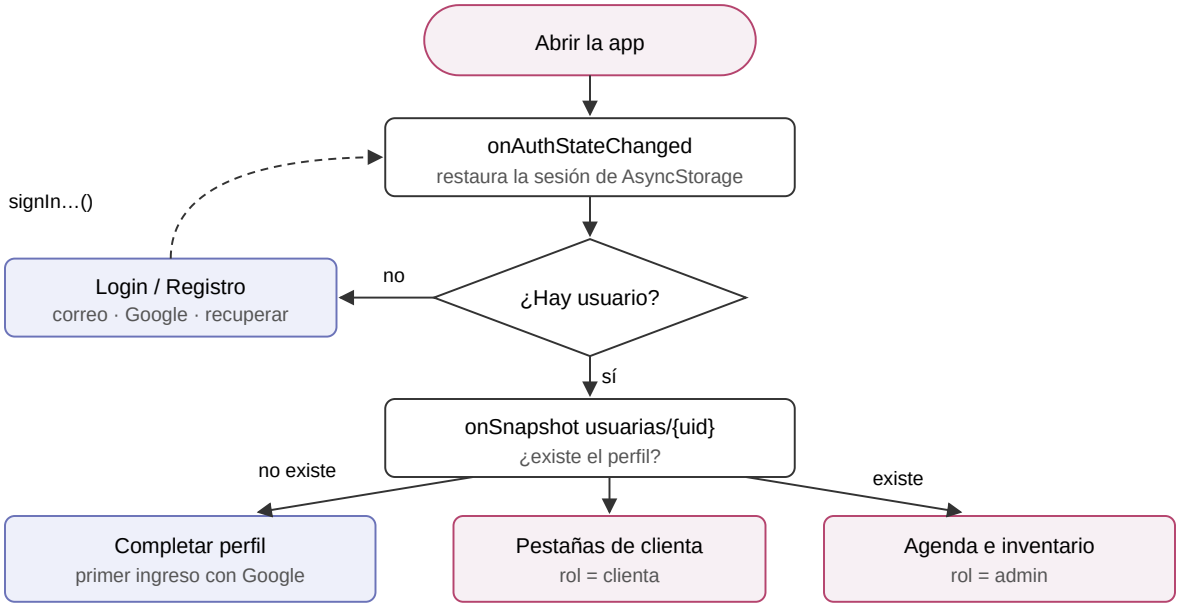
Hay otro detalle que no vi venir: Google entrega nombre y correo, pero no teléfono, y el salón lo necesita para confirmar citas. Cuando alguien entra con Google por primera vez, la sesión de Firebase existe pero el documento de perfil no. Para ese caso agregué la pantalla "Ya casi terminás", que pide el teléfono y el código de referido opcional antes de crear el perfil (Figura 3). Hasta que lo completa, la navegación no le muestra el resto de la app.

Sesión y Navegación

`AuthContext` se suscribe a `onAuthStateChanged` al abrir la app. Firebase restaura la sesión guardada en AsyncStorage y avisa; en ese momento me suscribo también al documento del perfil con `onSnapshot`. Con esas dos fuentes el contexto llega a uno de tres estados: sin sesión, perfil pendiente o sesión activa con su rol. El navegador raíz solo mira ese estado (Figura 2). No hay ninguna llamada a `navigate` después de iniciar sesión, y eso evita una clase entera de errores en los que la pantalla y la sesión quedan desincronizadas. Como el perfil también se escucha en tiempo real, si la administradora registra una inasistencia la clienta ve su contador actualizado sin cerrar la app.

Figura 2

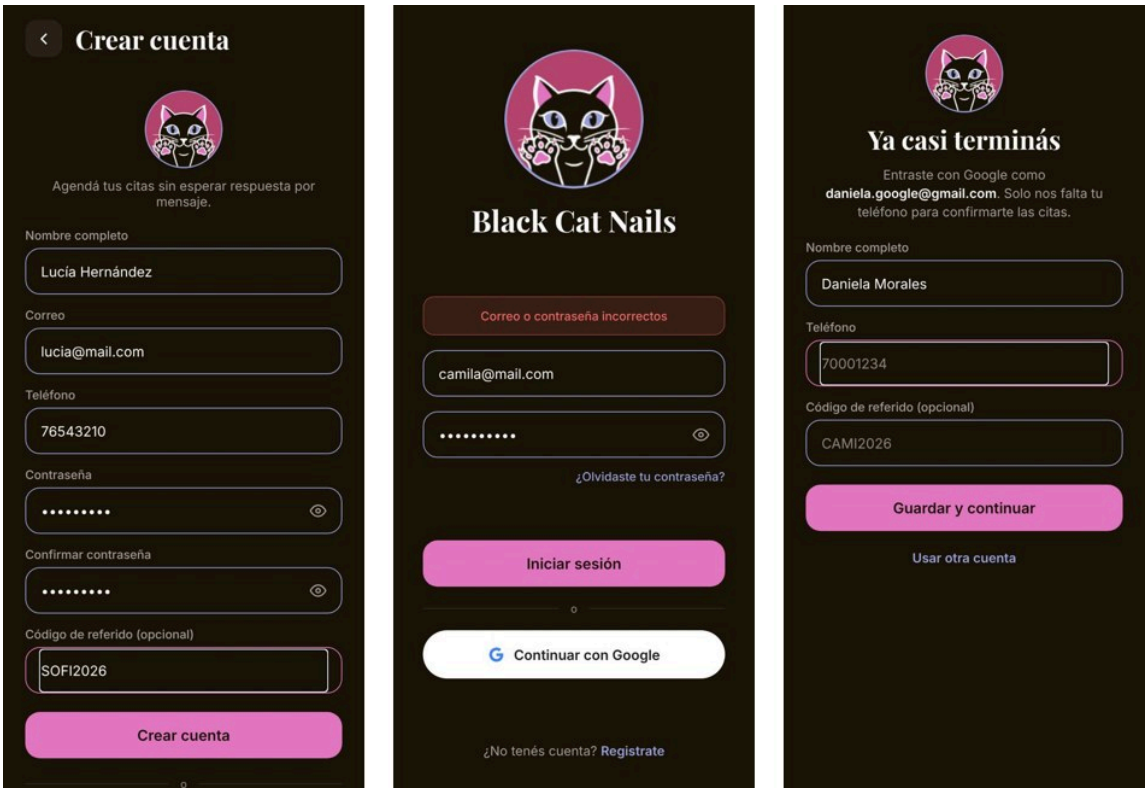
Flujo de autenticación y navegación según el estado de la sesión



Nota. Elaboración propia. La pantalla que se muestra depende solo del estado que entrega `AuthContext`; ninguna pantalla navega manualmente después de iniciar sesión.

Figura 3

Registro con código de referido, error de credenciales y primer ingreso con Google



Nota. Capturas propias de la aplicación conectada al emulador de Firebase, 5 de noviembre de 2026.

Modelo de Datos en Cloud Firestore

De Tablas a Documentos

En la Etapa 1 modelé los datos de forma relacional. Firestore no tiene joins, así que tuve que pensar al revés: en lugar de preguntar cómo normalizar, pregunté qué necesita leer cada pantalla y cuántas lecturas le cuesta. La documentación de Firebase recomienda justamente eso, elegir entre subcolecciones, colecciones raíz o datos anidados según las consultas que se van a hacer (Firebase, s.f.-d). La Tabla 2 resume cómo quedó.

Tabla 2

Colecciones de Firestore y su propósito

Ruta	Contenido	Quién lee / escribe
`usuarias/{uid}`	Nombre, correo, teléfono, rol, inasistencias, código de referido, foto	La dueña y la administradora
`usuarias/{uid}/favoritos/{id}`	Diseños guardados	Solo la dueña
`usuarias/{uid}/actividad/{id}`	Historial: tipo, descripción, fecha del servidor	La dueña agrega; nadie edita ni borra
`codigosReferido/{codigo}`	Índice de códigos únicos	Lectura puntual pública; se crea al registrarse
`referidos/{uid}`	Quién usó qué código (solo nombre)	La referente y la administradora
`servicios/{id}`	Catálogo con precio, duración y consumos	Todas leen; la administradora escribe
`insumos/{id}`	Inventario	Solo la administradora
`citas/{id}`	Cita con `clientaId`, servicios congelados, estado	La dueña y la administradora
`agendaDias/{fecha}`	Bloques ocupados del día, sin datos personales	Todas leen; cada clienta solo su bloque

Nota. Elaboración propia con base en `src/services/firebase/colecciones.ts` y `firestore.rules`.

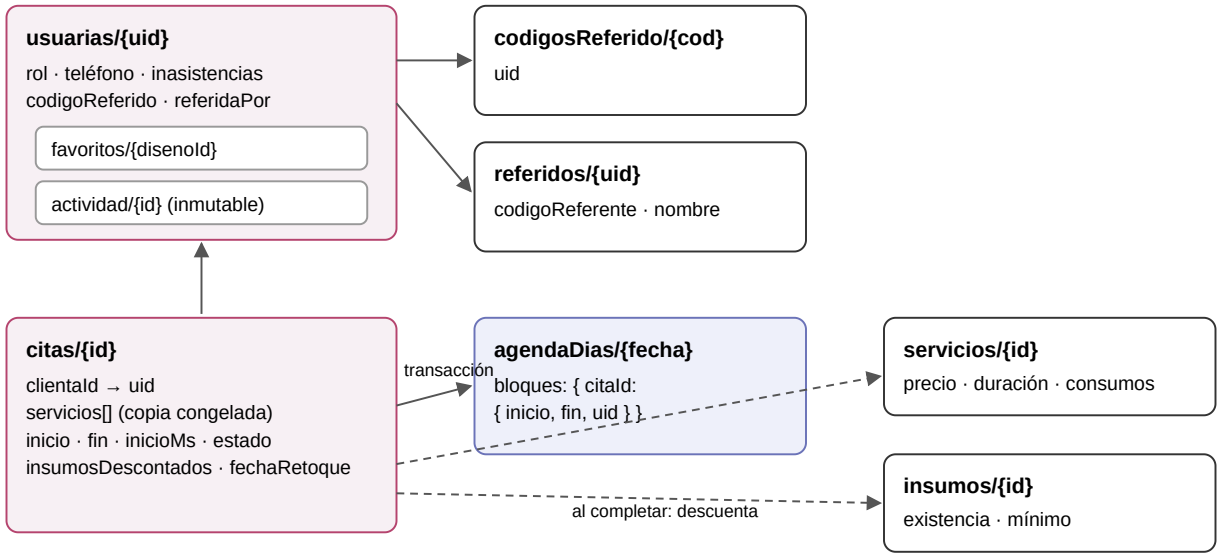
Tres decisiones merecen explicación. La primera: cada cita guarda una copia del nombre, precio y duración de sus servicios. Es la misma desnormalización de la Etapa 1, y en Firestore tiene una ventaja extra, porque la lista de citas se dibuja sin leer el catálogo. La segunda es la

subcolección de actividad. El historial pertenece a una sola usuaria y crece sin límite, así que vive debajo de su documento; si fuera un arreglo dentro del perfil, el documento pronto rozaría el límite de 1 MiB y cada lectura del perfil traería todo el historial.

La tercera es `agendaDias`, y es la que más pensé. Para ofrecer horarios libres, la app tiene que conocer las citas de todas las clientas de ese día. Pero una clienta no debería poder leer las citas de otra: ahí están su nombre, sus servicios y sus notas. Mi solución fue un documento por día con solo los bloques ocupados (inicio, fin y uid). La función `calcularHorarios` del dominio recibe esos bloques y devuelve los horarios disponibles, sin saber quién ocupa cada uno.

Figura 4

Estructura de datos en Firestore y relaciones entre colecciones



Nota. Elaboración propia. Las líneas continuas indican referencias por identificador; las punteadas, documentos que se modifican dentro de la misma transacción.

Escritura con Transacciones

En el Avance 1 la revalidación del horario era un `find` sobre un arreglo en memoria. Con varias clientas reservando desde teléfonos distintos eso ya no alcanza: dos podrían ver libre las 15:00 y confirmar al mismo tiempo. Crear una cita ahora es una transacción de Firestore que lee `agendaDias/{fecha}`, vuelve a calcular la disponibilidad con la misma función del dominio y, si

el horario sigue libre, escribe la cita y su bloque juntos. Si otra clienta modificó ese día en el medio, Firestore reintenta la transacción con los datos nuevos, y en el reintento la segunda clienta recibe el mensaje "Ese horario acaba de ocuparse" (Firebase, s.f.-e).

```
await runTransaction(db, async (tx) => {
  const dia = await tx.get(refAgendaDia(ymd));
  const bloques = bloquesDe(dia);
  const libre = calcularHorarios(ymd, duracion, Object.values(bloques))
    .find((h) => h.hora === hora)?.disponible;
  if (!libre) throw new ServiceError('horario_no_disponible', '…');
  tx.set(refNueva, { ...cita, actualizadaEn: serverTimestamp() });
  tx.set(refAgendaDia(ymd), { fecha: ymd, ultimaCita: cita.id,
    bloques: { ...bloques, [cita.id]: { inicio, fin, uid } } });
});
```

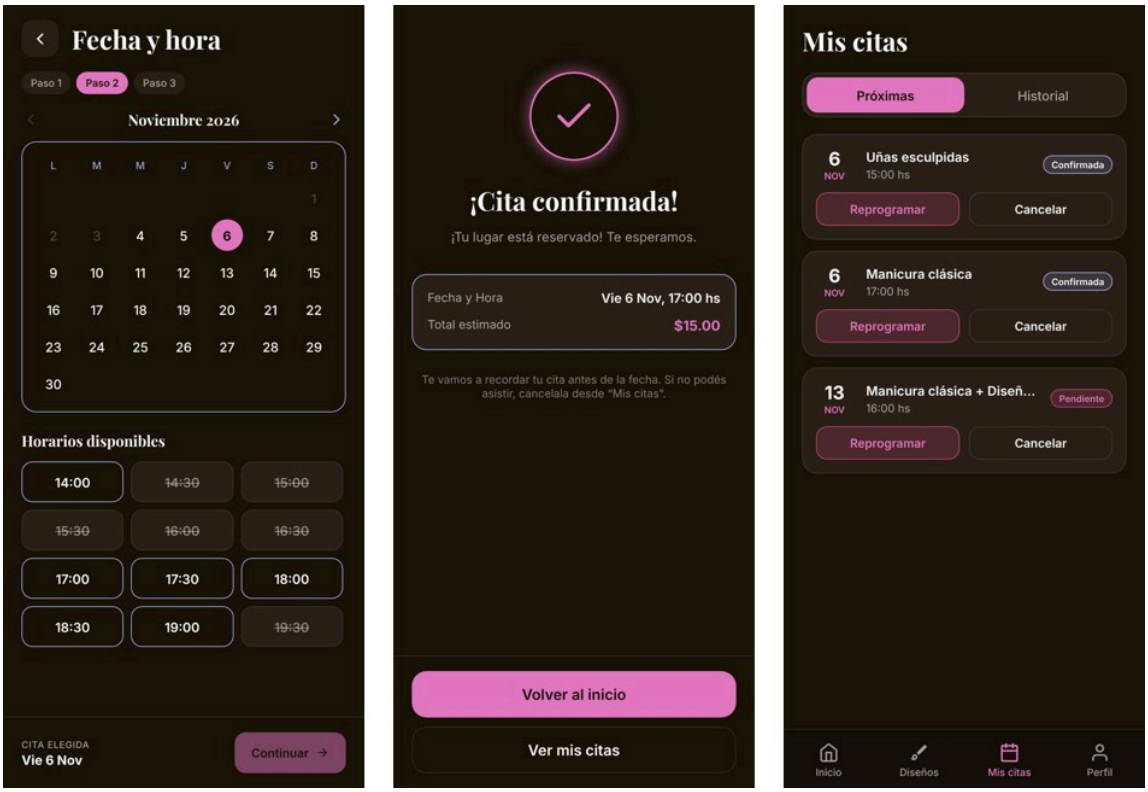
Cambiar el estado de una cita también es transaccional. Cuando la administradora marca una cita como completada, en una sola operación se actualiza el estado, se calcula la fecha de retoque, se descuentan los insumos de cada servicio y se registra el monto cobrado. Si es una inasistencia, se suma uno al contador de la clienta con `increment(1)`. O se guardan todos los cambios o ninguno, y el inventario nunca queda a medias.

Lectura en Tiempo Real

Las citas, el perfil, los favoritos, el historial y el inventario se leen con `onSnapshot`, que entrega los datos al suscribirse y cada vez que cambian en el servidor (Firebase, s.f.-e). Cada suscripción se abre en un `useEffect` y se cierra en su función de limpieza, así que al cerrar sesión no quedan oyentes vivos. Lo comprobé con dos navegadores abiertos: cuando la administradora inició la cita de Elena, en la pantalla de Elena la tarjeta pasó de "Confirmada" a "En curso" sin que tocara nada. Para la clienta la consulta es `where('clientaId', '==', uid)`, que coincide exactamente con lo que permiten las reglas; si pidiera todas las citas, Firestore rechazaría la consulta entera.

Figura 5

Disponibilidad calculada desde agendaDias, cita guardada y lista de próximas citas



Nota. Capturas propias. Los horarios tachados son bloques ocupados por otras clientas o que no alcanzan antes del cierre.

Reglas de Seguridad

La aplicación valida todo antes de escribir, pero eso no protege nada: cualquiera con la `apiKey` puede hablarle a Firestore sin pasar por la app. Por eso escribí las reglas como si la app no existiera. Las reglas de Firestore se evalúan en el servidor en cada lectura y escritura, y pueden consultar el documento existente, el documento que se quiere escribir y otros documentos con `get` y `getAfter` (Firebase, s.f.-i). La Tabla 3 resume lo que protege cada una.

Tabla 3

Reglas de seguridad principales

Riesgo	**Regla**
Una clienta se asigna el rol de administradora	Al crear el perfil, `rol` debe ser `clienta` e `inasistencias` 0; al editarlo, solo pueden cambiar teléfono, foto y correo
Borrarse las inasistencias	El campo no está entre los editables por la dueña
Leer citas ajenas	Lectura solo si `clientaId == request.auth.uid` o si es administradora
Crear una cita sin reservar el horario	`getAfter` exige que el bloque de esa cita exista en `agendaDias` al terminar la transacción
Liberar el horario de otra clienta	En `agendaDias` solo se puede agregar o quitar el bloque propio, y quitarlo exige que la cita quede cancelada
Cancelar a última hora	`inicioMs - request.time` debe ser de al menos 12 horas
Falsear o borrar el historial	`fecha == request.time` (hora del servidor); `update` y `delete` prohibidos
Apropiarse de un código de referido	El código se crea una sola vez y debe coincidir con el del perfil escrito en el mismo batch
Ver el inventario o cambiar precios	`insumos` y la escritura de `servicios` solo para administradora

Nota. Elaboración propia con base en `firestore.rules`.

La regla de las doce horas es un buen ejemplo de por qué tuve que agregar campos que la app no necesitaba. Las reglas no saben interpretar una fecha ISO como texto, así que cada cita guarda también `inicioMs`, el inicio en milisegundos, para poder compararlo con `request.time`. Lo mismo pasa con `ultimaCita` en `agendaDias`: las reglas no pueden recorrer un mapa, así que la app declara qué bloque está tocando y la regla verifica que sea el único que cambió.

Las reglas que no se prueban fallan sin aviso, así que escribí 37 casos con `@firebase/rules-unit-testing` que corren contra el emulador (`npm run test:reglas`). Cada caso simula a una clienta, a otra clienta, a la administradora o a alguien sin sesión, e intenta tanto operaciones válidas como ataques. Las 37 pasan. Uno de ellos falló la primera vez y me enseñó algo: probé que una clienta no pudiera poner sus inasistencias en cero, pero la usuaria de prueba ya tenía cero, así que la escritura no cambiaba nada y Firestore la permitía. La regla estaba bien; el caso estaba mal planteado.

Módulos Secundarios e Historial de Registros

Mi Actividad

El Foro 2 pide una forma de ver los registros almacenados. Ya existía el historial de citas en "Mis citas", pero quería algo que mostrara todo lo que la clienta hizo con su cuenta. Cada acción importante (registrarse, iniciar sesión, agendar, cancelar, cambiar el teléfono o la foto, guardar un favorito) agrega un documento a `usuarias/{uid}/actividad` con la hora del servidor. Cuando la administradora cambia el estado de una cita, el registro aparece en el historial de la clienta, por ejemplo "Tu cita del Vie 6 Nov · 17:00 pasó a Confirmada". La pantalla agrupa los registros por día, permite filtrar entre citas y cuenta, y arriba muestra cuántas citas tiene, cuántas completó y cuánto lleva invertido (Figura 6).

Registrar la actividad nunca bloquea la acción principal. Si el registro falla por la red, la cita igual se guarda. Me pareció la prioridad correcta: perder una línea del historial molesta, pero perder una cita es un problema para el negocio.

Favoritos, Referidos y Foto de Perfil

En el Avance 1 los favoritos se guardaban en el teléfono. Ahora viven en `usuarias/{uid}/favoritos`, así que la clienta los conserva si cambia de teléfono o entra desde la web. Al tocar el corazón la interfaz cambia de inmediato y, si Firestore rechaza la escritura, se revierte. Los referidos dejaron de calcularse recorriendo todas las usuarias, cosa que las reglas ya no permiten, y ahora se consultan en `referidos`, que solo guarda el nombre de quien usó el código.

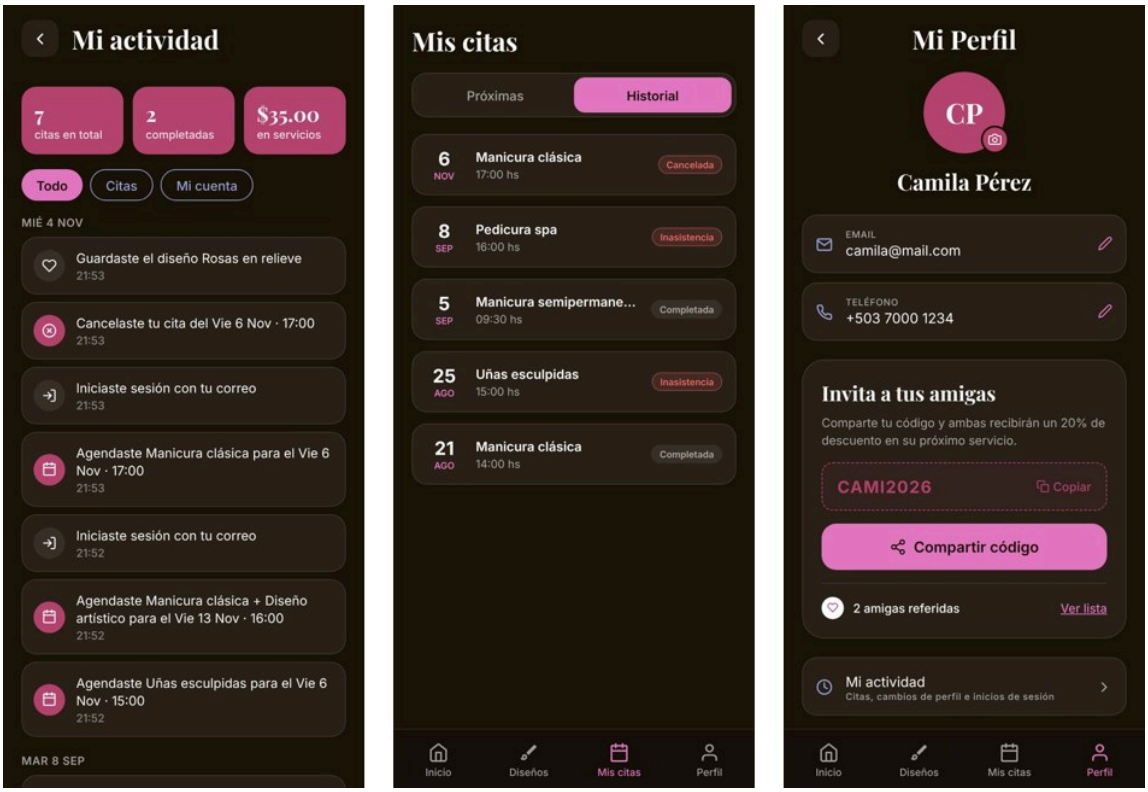
Las imágenes me obligaron a cambiar de plan. Quería usar Cloud Storage, pero en los proyectos nuevos Storage exige el plan Blaze, que pide tarjeta de crédito (Firebase, s.f.-h). Para no depender de eso, la foto de perfil se reduce a 320 píxeles y la imagen de referencia de la cita a 600 con `expo-image-manipulator`, y se guardan como JPEG en base64 dentro del documento. Pesan entre 25 y 80 KB, lejos del límite de 1 MiB. No es lo ideal para una galería grande, pero para una foto por clienta funciona y se ve igual en todos sus dispositivos.

Inventario de la Administradora

La administradora tiene una pestaña nueva de inventario que escucha la colección `insumos`. Cada insumo muestra su existencia, una barra con semáforo (suficiente, bajo o crítico, con la función `calcularEstado` de la web) y para cuántas clientas alcanza según el consumo promedio de los servicios. Al completar una cita en la agenda, la transacción descuenta los insumos y la cifra baja en la otra pestaña sin recargar. En la prueba de la Figura 7, completar una pedicura bajó el removedor de 40 a 30 ml.

Figura 6

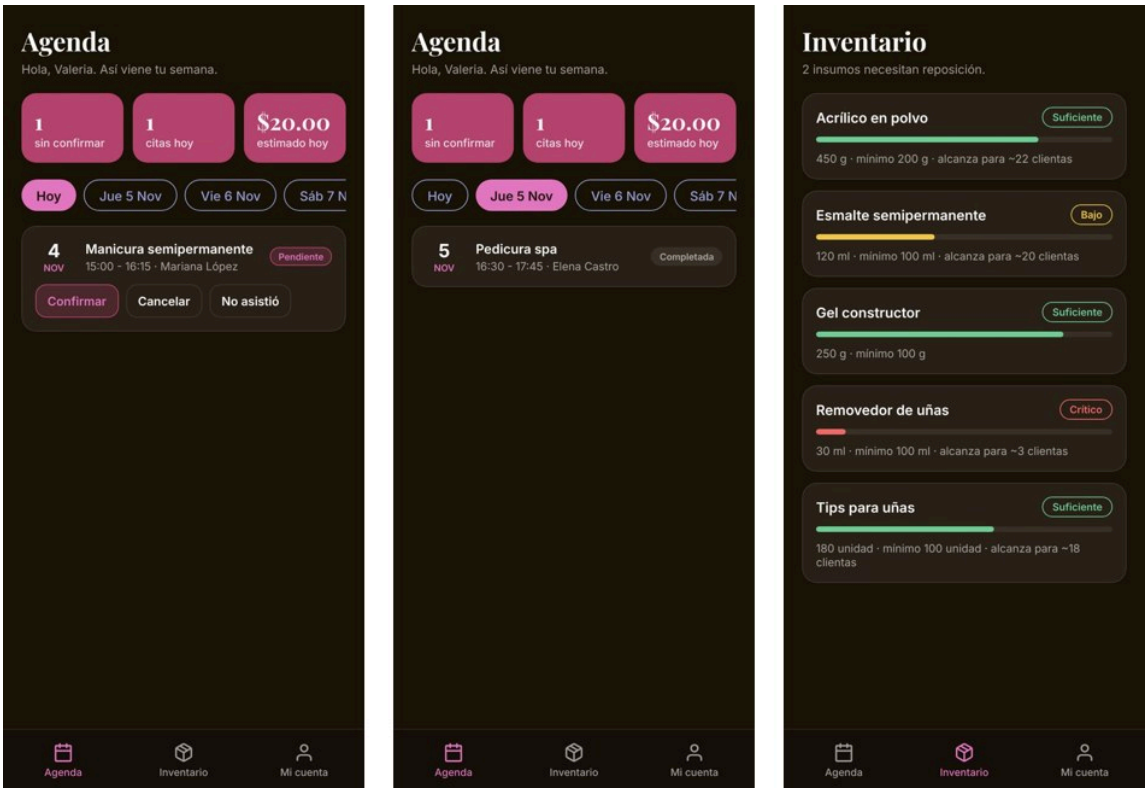
Historial de actividad, historial de citas y perfil con referidas



Nota. Capturas propias. "Mi actividad" se actualiza en tiempo real; la cita cancelada aparece en ambos historiales.

Figura 7

Agenda de la administradora antes y después de completar una cita, e inventario actualizado



Nota. Capturas propias. El removedor de uñas pasó de 40 ml a 30 ml en la misma transacción que completó la cita.

Pruebas Realizadas

Probé en tres niveles. TypeScript compila sin errores con `npm run typecheck`. Las reglas pasan los 37 casos del emulador. Y recorrí los flujos completos en la versión web de la app, conectada a los emuladores de Authentication y Firestore con los datos que carga `npm run seed:emuladores`. La Tabla 4 resume los casos funcionales.

Tabla 4

Casos de prueba funcionales del Avance 2

#	Caso	Resultado esperado	Resultado
1	Login con contraseña incorrecta	"Correo o contraseña incorrectos", sin revelar si el correo existe	Correcto
2	Login de clienta y de administradora	Cada rol ve sus pestañas; los datos llegan de Firestore	Correcto
3	Registro con código SOFI2026	Cuenta en Auth, perfil y código propio en Firestore, referida registrada	Correcto
4	Registro con código inexistente	"El código de referido no existe"; no se crea la cuenta	Correcto
5	Primer ingreso con Google	Pantalla para completar teléfono; luego inicio de clienta	Correcto*
6	Agendar el viernes con citas existentes	Horarios ocupados tachados; cita y bloque guardados juntos	Correcto
7	Cancelar con más de 12 horas	Cita cancelada, bloque liberado, registro en actividad	Correcto
8	Guardar un diseño como favorito	Documento en `favoritos` y registro en actividad	Correcto
9	Administradora completa una cita	Estado, retoque e inventario actualizados en una transacción	Correcto
10	Cambio hecho por la administradora	La clienta lo ve sin recargar (onSnapshot)	Correcto
11	Cerrar y abrir la app con sesión activa	Firebase restaura la sesión sin pedir credenciales	Correcto

Nota. Elaboración propia. Pruebas del 5 de noviembre de 2026. *El caso 5 se probó primero con una credencial de Google de prueba del emulador y el 8 de noviembre con una cuenta real de Google en la compilación de iOS (simulador de iPhone), contra el proyecto de Firebase.

Análisis del Uso de Firebase en React Native

Lo que más me convenció fue lo poco que tuvo que cambiar el resto de la app. Las pantallas siguen hablando con los mismos tres contextos, y los contextos con los mismos servicios. Cambié lo que hay dentro de `services/` y agregué suscripciones en los contextos, pero el diseño y la lógica de negocio no se tocaron. La separación en capas que planteé en la Etapa 1 pagó lo que prometía.

Firebase también me obligó a pensar en seguridad de una manera que la API simulada nunca exigió. En el Avance 1 la validación de rol estaba en el servicio, que corre en el teléfono de la clienta. Era una validación de mentira. Escribir las reglas me hizo preguntarme, colección por colección, qué pasa si alguien ignora la app, y de esas preguntas salió `agendaDias`, que no estaba en ningún diagrama.

Hay cosas que no me terminan de gustar. La primera es que Google en el teléfono no funciona en Expo Go, y eso rompe la idea de "escaneá el QR y probá". La segunda es la caché sin conexión: el SDK web no persiste Firestore en React Native, así que sin internet la app muestra lo que tiene en memoria pero no sobrevive a un cierre. React Native Firebase resolvería ambas, a cambio de compilar siempre. La tercera es más de fondo: el negocio queda atado a Google. Exportar datos de Firestore es posible, pero migrar las reglas y el modelo a otra base no sería trivial. Para un salón pequeño me parece un costo aceptable frente a no tener que mantener un servidor, pero vale dejarlo dicho.

Y una decisión de la que todavía no estoy seguro: validar la disponibilidad en el cliente dentro de una transacción. Funciona, y las reglas garantizan que nadie se quede con un bloque ajeno, pero las reglas no pueden verificar que el bloque nuevo no se cruce con otro. Una clienta que manipule la app podría reservar un bloque encimado. Con Cloud Functions esa validación iría en el servidor, pero también requieren el plan Blaze. Por ahora la administradora ve cualquier choque en su agenda, y lo dejo como recomendación.

Conclusiones y Recomendaciones

Conclusiones

La aplicación ya cumple su función con datos reales. Las cuentas viven en Firebase Authentication, con correo, Google y recuperación de contraseña, y la sesión se mantiene entre aperturas. Todo lo que la clienta hace queda en Firestore asociado a su uid, y la administradora lo ve en tiempo real desde su teléfono.

El modelo de datos salió de las consultas y no de las entidades. Pensar qué lee cada pantalla me llevó a guardar copias de los servicios en la cita, a separar el historial en una subcolección y a crear `agendaDias` para publicar la disponibilidad sin exponer a nadie.

Las transacciones resolvieron un problema que el diagnóstico de la Etapa 1 señalaba desde el inicio, las citas encimadas. Antes era una validación local; ahora es una operación atómica contra el servidor.

Y las reglas de seguridad fueron lo que más tiempo me llevó y lo que más aprendí. Probarlas con casos de ataque, y no solo con el camino feliz, encontró un caso mal planteado que de otra forma habría pasado como prueba exitosa.

Recomendaciones

- Conectar la versión web de la Etapa 2 al mismo proyecto de Firebase antes de la entrega final, reutilizando los servicios de este avance, para que la persistencia sea compartida entre web y móvil como pide el proyecto.
- Mover la validación de choques de horario a una Cloud Function cuando el negocio pueda pasar al plan Blaze, y con eso también llevar las imágenes a Cloud Storage.
- Evaluar React Native Firebase en la versión final si la caché sin conexión resulta necesaria para las clientas.
- Agregar notificaciones push con Firebase Cloud Messaging para el recordatorio de cita y el aviso de retoque.

Referencias

Expo. (s.f.-a). *Using Firebase*. Expo Documentation. https://docs.expo.dev/guides/using-firebase/

Expo. (s.f.-b). *Google authentication*. Expo Documentation. https://docs.expo.dev/guides/google-authentication/

Expo. (s.f.-c). *Environment variables in Expo*. Expo Documentation. https://docs.expo.dev/guides/environment-variables/

Firebase. (s.f.-a). *Firebase documentation*. Google. https://firebase.google.com/docs

Firebase. (s.f.-b). *Firebase Authentication*. Google. https://firebase.google.com/docs/auth

Firebase. (s.f.-c). *Cloud Firestore data model*. Google. https://firebase.google.com/docs/firestore/data-model

Firebase. (s.f.-d). *Choose a data structure*. Google. https://firebase.google.com/docs/firestore/manage-data/structure-data

Firebase. (s.f.-e). *Transactions and batched writes*. Google. https://firebase.google.com/docs/firestore/manage-data/transactions

Firebase. (s.f.-f). *Introduction to Firebase Local Emulator Suite*. Google. https://firebase.google.com/docs/emulator-suite

Firebase. (s.f.-g). *Authenticate using Google with JavaScript*. Google. https://firebase.google.com/docs/auth/web/google-signin

Firebase. (s.f.-h). *Firebase pricing*. Google. https://firebase.google.com/pricing

Firebase. (s.f.-i). *Get started with Cloud Firestore Security Rules*. Google. https://firebase.google.com/docs/firestore/security/get-started

García Villeda, E. A. (2026a). *Black Cat Nails Web: Análisis y planificación de una aplicación multiplataforma para la gestión de un emprendimiento de servicios de manicura* [Trabajo de cátedra no publicado]. Universidad Don Bosco.

García Villeda, E. A. (2026b). *Black Cat Nails Móvil: React Native y Figma aplicados al diseño e implementación de la aplicación para clientas de un emprendimiento de manicura* [Trabajo de cátedra no publicado]. Universidad Don Bosco.

Moroney, L. (2017). *The definitive guide to Firebase: Build Android apps on Google's mobile platform*. Apress. https://doi.org/10.1007/978-1-4842-2943-9

OWASP Foundation. (2024). *OWASP Mobile Top 10*. https://owasp.org/www-project-mobile-top-10/

React Native Google Sign In. (s.f.). *Google Sign In for React Native*. https://react-native-google-signin.github.io/docs/

Román Gallardo, A., Flores Cortés, C. A., & Herrera Morales, J. R. (s.f.). *Programación de aplicaciones móviles con React Native*. Editorial BUK. https://cbues.bibliotecasdigitales.com/

Apéndice

Enlaces del Avance

Repositorio: https://github.com/Kikeagv/black-cat-nails-mobile (rama enrique-garcia)

Reglas de seguridad: firestore.rules en la raíz del repositorio

Video de demostración: https://youtu.be/A5dQKUByHSs

Cuentas de Demostración

Clienta: camila@mail.com · Clienta con retoque vencido: sofia@mail.com · Administradora: vante@blackcatnails.sv. Contraseña de las tres cuentas: Password123.

Comandos del Proyecto

Comando	Qué hace
`npm run seed`	Crea las cuentas de prueba y carga Firestore (usa la cuenta de servicio)
`npm run reglas:deploy`	Publica `firestore.rules` en el proyecto
`npm run emuladores`	Levanta Authentication y Firestore en local
`npm run test:reglas`	Corre los 37 casos de las reglas de seguridad
`npx expo run:ios`	Compila la app para el simulador con Google Sign-In